Seamless Pattern Tile Pipeline: Full Project Report

Oct 24, 2026 · @A.J

Summary

We built a Python pipeline that checks whether a repeat tile is seamless and, when it is not, asks an image model to repair only the bad join spots. The first four stages (tiling, diamond markings, seam check, return the original) work and were confirmed correct. The AI repair stage has been rewritten three times and is not yet confirmed on real tiles.

- **Aim:** take one design tile, decide objectively whether it repeats without a visible seam, and if not, return a repaired tile of the same size that reuses only the original design.
- **Confirmed working:** `tile_2x2.py`, `draw_diamond.py` and `check_seamless.py`, including returning the original image when it is seamless.
- **First AI repair (GPT):** passed the seam test but left overlapping, ghosted motifs along the joins.
- **Second version (v2):** still failed the test and showed rectangular patch blocks. The cause was a threshold bug in our own code, not the prompt.
- **Third version (v3) and the combined GPT/Gemini script:** fix that cause. They have been tested only against mock API clients, not yet against the real models.
- **Next:** run the latest script on real tiles with each model and tune the thresholds.

Background and aim

The project is a seamless (repeat-tile) image generator for print designs such as textile and wallpaper patterns. A tile is seamless when copies placed edge to edge show no visible line, gap or broken motif at the joins. The work in this report took place on 23 and 24 October 2026.

Rules for the output (set by the user)

- The output must be the same size as the input.
- Only designs, motifs and colours from the original image may appear. Nothing new or "out of the box".
- Print quality, no watermarks, and no changes outside the repaired area.

The wider plan is to classify each design as easy, medium or hard, pick a model per type, and add a chat-style edit screen later. This report covers only the tile-checking and repair scripts built so far.

Working environment

Item	Detail
Computer	Windows, PowerShell inside VS Code, folder `C:\Users\aryap\Desktop\version1`
Python	Virtual environment named `venv`
Libraries	Pillow, numpy, scipy, openai, google-genai
Image models	GPT: `gpt-image-2.5-flare` (OpenAI). Gemini: `gemini-nano-banana-2.1`
Test designs	A paisley tile (2268 x 2268 px), then a blue floral and a watercolour design shared later

Timeline of stages

The work ran in eight stages, each building on the one before. Stages 1 to 4 were confirmed correct by running them; stages 5 to 8 are the AI repair work.

#	Stage	Script	Outcome
1	Arrange a tile in a 2x2 grid with no gaps	`tile_2x2.py`	Correct, confirmed
2	Draw a diamond on the 2x2 result, then add dotted diagonals	`draw_diamond.py` (two versions)	Correct, confirmed
3	Check whether the joins are seamless	`check_seamless.py`	Correct on test and flower tiles; threshold tuned to 1.1
4	Return the original image when it is seamless	`check_seamless.py` update	Correct, confirmed

#	Stage	Script	Outcome
5	Repair non-seamless tiles with the GPT image model	`fix_seamless.py` v1	Passed the seam test, but motifs overlapped and ghosted
6	Remove overlap and place patches cleanly	`fix_seamless.py` v2	Still not seamless; a threshold bug in our code was found
7	Narrow-band repair, one image per spot	`fix_seamless.py` v3	Tested with mock API clients only
8	Choose GPT or Gemini at run time	`seamless_gemini.py`	Tested with mock API clients only; real runs pending

Stage 1: Tile the image 2x2

The first script, `tile_2x2.py`, places four copies of an image in a 2x2 grid with no overlap and no gap, and the user confirmed the output was correct.

Request. Take an image as input, duplicate it, and arrange the copies in a 2x2 grid, joined precisely with no line gap between them.

Method.

1. Create a blank canvas exactly twice the width and height of the input.
2. Paste the image four times at offsets (0, 0), (w, 0), (0, h) and (w, h). Because each offset equals the tile size, neighbouring tiles touch with no gap and no overlap.
3. Keep transparency if the image has it. JPEG output is saved at quality 100.

How it was run (Windows, inside the virtual environment).

```
python -m venv venv
venv\Scripts\activate
pip install pillow
python tile_2x2.py pattern.png
```

The output is saved next to the input as `pattern_2x2.png`; `-o name.png` sets another name. PNG and JPG inputs both work. If `python` is not recognised, `py` (Windows) or `python3` (Mac/Linux) can be used. In PowerShell, if activation is blocked, `Set-`

`ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` fixes it for that session.



Figure 1. The user's sketch of the goal: one tile (left) and the 2x2 arrangement (right).

Result. The user ran it and reported the output was what they wanted. The thin dark lines at the joins in the sketch come from dark border pixels inside the tile itself, which the script copies as they are. Trimming 1 or 2 pixels from each tile edge would remove them; a tile that is itself seamless joins perfectly.

Stage 2: Diamond markings

`draw_diamond.py` marks a diamond whose corners are the midpoints of the four image edges, and in its second version adds two dotted diagonals through the centre. The user confirmed the output of both versions.

Request. Take the 2x2 result as input and draw diamond-like markings on it. The dotted lines in the user's sketch only show where the tiles were joined, so they must not be printed in the output.



Figure 2. The 2x2 result, the preview with the join drawn as dotted lines for explanation only, and the diamond to be drawn.

Version 1: diamond only.

- A solid line joins the midpoints of the top, right, bottom and left edges.
- The line is drawn on a layer at four times the resolution and scaled down, so the diagonals are smooth rather than jagged.
- Options: `-w` line width (default scales with image size), `-c` colour (default black), `-o` output name.

Version 2: dotted diagonals added. The user then asked for the diagonals of the diamond, meaning the vertical and horizontal dotted lines through the centre.

- Round dots run top to bottom and left to right, joining opposite corners of the diamond.
- Dot size equals the line width, so `-w` changes both. `--dot-gap` (default 3) sets the spacing and `--no-dotted` draws the diamond only.

```
python draw_diamond.py pattern_2x2.png
```

The output is saved as `pattern_2x2_diamond.png`. The left side of Figure 3 in the next stage shows the confirmed result.

Stage 3: Seam checker

Yes, a Python script can judge whether a design repeats seamlessly. `check_seamless.py` measures the colour jump across the tile join and compares it with how much the image changes between neighbouring pixels elsewhere. It is a measurement, not a proof, and the user confirmed it gave correct answers on their test images.

Request. Check the design along the diagonals, up-down and left-right (the blue boxes in the user's sketch), and answer "yes, the design pattern is seamless" or "no, the design is not seamless".

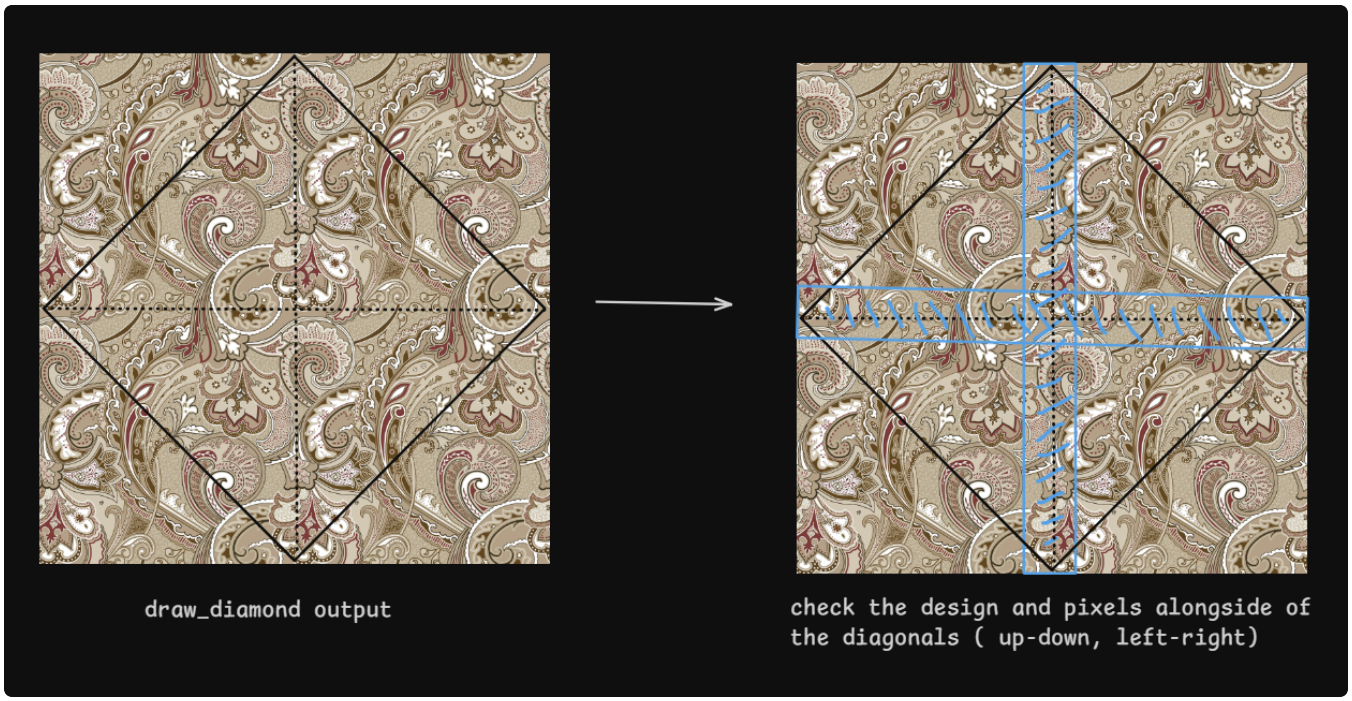


Figure 3. Left: the confirmed diamond output. Right: the blue boxes mark the pixels and designs to be compared along the up-down and left-right joins.

Method.

1. Take the clean 2x2 image (no diamond or dotted lines drawn on it, since drawn lines would count as a mismatch).
2. For every row along the up-down join, measure the average colour difference between the pixel just left of the join and the pixel just right of it. This is the seam jump. Do the same for every column along the left-right join.
3. Measure the normal pixel-to-pixel difference in the same rows elsewhere in the image. This is the baseline. It is never allowed below 1.0, so flat areas do not give huge ratios.
4. Ratio = seam jump divided by baseline. About 1.0 or lower means the join is no more visible than any other part of the image.
5. Split each join into 20 sections and score each one, so a mismatch in a small area is still caught.
6. Pass only if the overall ratio is at or below the threshold and the worst section is at or below twice the threshold. The default threshold is 1.5.

Reading the printout.

- **Overall ratio:** the average over the whole join line.
- **Worst section:** the highest of the 20 section ratios.
- **Positions:** the pixel range of that worst section. For the up-down join these are rows (y values); for the left-right join they are columns (x values).
- **Odd width/height warning:** the centre line is then not an exact tile join. If the file is a single tile, run it with `--single-tile`.

Options. `--single-tile` tiles the input 2x2 internally. `--threshold` sets strictness (lower is stricter). `--segments` sets the number of sections. `--debug` saves a copy with failing sections marked red, for example `python check_seamless.py test.jpg --single-tile --debug` creates `test_seamcheck.jpg`.

Real runs (from the user's terminal).

File	Join	Overall ratio	Worst section (positions)	Result
`test.jpg`	Up-down	0.81	3.22 (1010-1062)	Mismatch
`test.jpg`	Left-right	1.48	3.84 (675-716)	Mismatch
`flower.jpeg`	Up-down	0.29	0.80 (419-445)	OK
`flower.jpeg`	Left-right	0.41	1.51 (473-499)	OK

`test.jpg` was reported as not seamless, even though its overall ratios passed, because its worst sections exceeded the limit of 3.0. `flower.jpeg` passed both joins and was reported as seamless.

```
(venv) PS C:\Users\aryap\Desktop\version1> python check_seamless_v2.py test.jpg
Warning: image width/height is odd, so the centre line is not an exact tile join.
Up-down seam   : overall ratio 0.81, worst section 3.22 (positions 1010-1062) -> MISMATCH
Left-right seam: overall ratio 1.48, worst section 3.84 (positions 675-716) -> MISMATCH

NO - the design pattern is not seamless.
(venv) PS C:\Users\aryap\Desktop\version1> python check_seamless.py flower.jpeg
Up-down seam   : overall ratio 0.29, worst section 0.80 (positions 419-445) -> OK
Left-right seam: overall ratio 0.41, worst section 1.51 (positions 473-499) -> OK

YES - the design pattern is seamless.
```

Figure 4. The seam checker run on two images.

Threshold. The user set the threshold to 1.1 and found it gave the correct results. Changing the threshold needs no other change, because the worst-section limit is always set to twice the threshold (2.2 at 1.1).

Stage 4: Return the original when seamless

When the check passes, `check_seamless.py` now saves an untouched copy of the original image, and the user confirmed this worked.

Request. If the image is seamless, return the original image that was used as input.

Method.

- The copy is byte for byte (`shutil.copyfile`), so there is no re-encoding and no quality loss.
- It is saved as `<name>_seamless.<ext>` next to the original. `-o name.png` sets another name or place.
- `--original path` returns a different file from the one checked, for example the single tile while the 2x2 result is checked.
- If the design is not seamless, nothing is saved at this stage.

```
python check_seamless.py pattern.png --single-tile
python check_seamless.py pattern_2x2.png --original pattern.png
```

"The original image" was read as the single tile. If the 2x2 result is wanted instead, run without `--original`.

Stage 5: First AI repair with the GPT image model

The first repair script, `fix_seamless.py` v1, sent the join area to `gpt-image-2.5-flare` for repainting. Its output passed the seam test, but the repaired areas showed overlapping motifs (Stage 6).

Request. When the checker says the image is not seamless, call an image model (`gpt-image-2.5-flare`, with the API key left as a placeholder for the user to fill in). The model must make the border areas seamless by reusing designs from the original image or aligning mismatched patterns, must not bring in designs from outside the original, and must return an image the same size as the input.

What was checked first. A web search showed that GPT Image 2.5 was released on 8 September 2026 in two API versions, Flare (fast, the default) and Sunburst (higher precision). The edit endpoint accepts an optional mask, and transparent mask pixels mark the area to change. One provider documents that the mask is guidance rather than a hard boundary, so edits can spill past its shape. The model is also sold through several gateways with different request formats, so the script calls OpenAI's own `images.edit` directly. The accepted image sizes and exact mask format could not be confirmed in advance; the user's later runs showed the calls worked.

Method (v1).

1. Run the same seam test as the checker. If seamless, save a copy of the original and stop.
2. Build a wrapped canvas: the tile repeated 2x2, with a window cut out so the tile's own edges meet in the centre, plus extra context (a quarter of the tile) on every side.
3. Draw a cross-shaped mask over the join, 15% of the tile wide by default. Transparent means repaint.
4. Send canvas, mask and prompt to the model (longest side capped at 2048 px). Resize the answer back to the canvas size.
5. Paste only the cross back onto the untouched canvas with a soft edge, cut the tile back out at the original size, and run the seam test again.
6. If it still fails, retry up to two more times, each with a strip 10 points wider.

Setup. `pip install openai numpy pillow`, then put the key in `API_KEY`. Run with `python fix_seamless.py flower.jpeg`.

First error: FileNotFoundError. The first run failed with "No such file or directory: 'flower.jpg'". The earlier checker run had used `flower.jpeg`, so the script was looking for the wrong extension. Using the exact file name, or the full path in quotes, fixes it. `dir` lists the file names in the folder.

```
(venv) PS C:\Users\aryap\Desktop\version1> python fix_seamless.py flower.jpg
Traceback (most recent call last):
  File "C:\Users\aryap\Desktop\version1\fix_seamless.py", line 218, in <module>
    sys.exit(main())
             ~~~~^^
  File "C:\Users\aryap\Desktop\version1\fix_seamless.py", line 170, in main
    img = Image.open(src).convert("RGB")
          ~~~~~~~~~~~~~~^^^^^
  File "C:\Users\aryap\Desktop\version1\venv\Lib\site-packages\PIL\Image.py", line 3654, in open
    fp = builtins.open(filename, "rb")
FileNotFoundError: [Errno 2] No such file or directory: 'flower.jpg'
(venv) PS C:\Users\aryap\Desktop\version1>
```

Figure 5. FileNotFoundError: the file was named flower.jpeg, not flower.jpg.

Result. After the name was corrected, the user reported the output was reported as seamless by the checker. The next problem was visual quality, described in Stage 6.